Programming Assignment Report

Design and Analysis of Algorithms

Name (as in ERP): Poornima H A

Registration Number (in full): 2411021061248

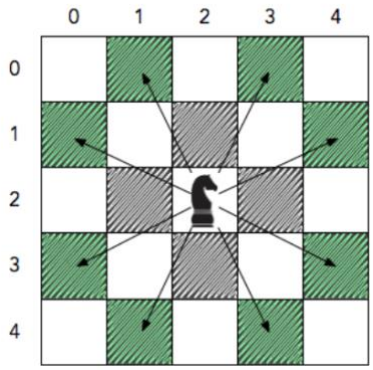
Problem Name: Kingts on Chessboard

Problem Statement:

KnightL is a chess piece that moves in an **L shape**. We define the possible moves of KnightL(a, b) as any movement from some position (x1,y1)(x_1, y_1)(x1,y1) to some (x2,y2)(x_2, y_2)(x2,y2) satisfying either of the following:

- x2=x1±ax_2 = x_1 \pm ax2=x1±a and y2=y1±by_2 = y_1 \pm by2=y1±b, or
- x2=x1±bx_2 = x_1 \pm bx2=x1±b and y2=y1±ay_2 = y_1 \pm ay2=y1±a

Note that **(a, b)** and **(b, a)** allow for the same exact set of movements. For example, the diagram below depicts the possible locations that KnightL(1, 2) or KnightL(2, 1) can move to from its current location at the center of a 5 × 5 chessboard**.**



Observe that for each possible movement, the Knight moves 2 units in one direction (i.e., horizontal or vertical) and 1 unit in the perpendicular direction**.**

Given the value of **n** for an **n × n chessboard**, answer the following question for each **(a, b)** pair where **1 ≤ a, b < n**:

- What is the **minimum number of moves** it takes for **KnightL(a, b)** to get from position **(0, 0)** to position **(n − 1, n − 1)**? If it's not possible for the Knight to reach that destination, the answer is **−1** instead.

Then print the answer for each **KnightL(a, b)** according to the **Output Format** specified below.

Input Format

A single integer denoting **n**.

Constraints

- **$5 \leq n \leq 25$**

Output Format

Print exactly **$n - 1$** lines of output in which each line **i** (where **$1 \leq i < n$**) contains **$n - 1$ space-separated integers** describing the minimum number of moves **KnightL(i, j)** must make for each respective **j** (where **$1 \leq j < n$**).

If some **KnightL(i, j)** cannot reach position **$(n - 1, n - 1)$**, print **−1** instead.

For example, if **n = 3**, we organize the answers for all the **(i, j)** pairs in our output like this:

(1,1) (1,2)

(2,1) (2,2)

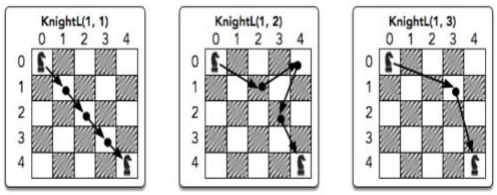
Problem HackerRank Link:

https://www.hackerrank.com/challenges/knightl-on-chessboard/problem

Github Submission Link:

https://github.com/Poornima-HA/DAA_LAB/blob/main/Assignment_Hackerrank

Solution Steps: The diagram below depicts possible minimal paths for the following:-



One minimal path for **KnightL(1,4)** is:

(0,0) → (1,4) → (2,0) → (3,4) → (4,0) → (0,1) → (4,2) → (0,3) → (4,4)

We then print:

4 4 2 8

as our first line of output because:

- KnightL(1,1) took 4 moves
- KnightL(1,2) took 4 moves
- KnightL(1,3) took 2 moves
- KnightL(1,4) took 8 moves

In some of the later rows of output, it's impossible for **KnightL(i,j)** to reach position **(4,4)**.

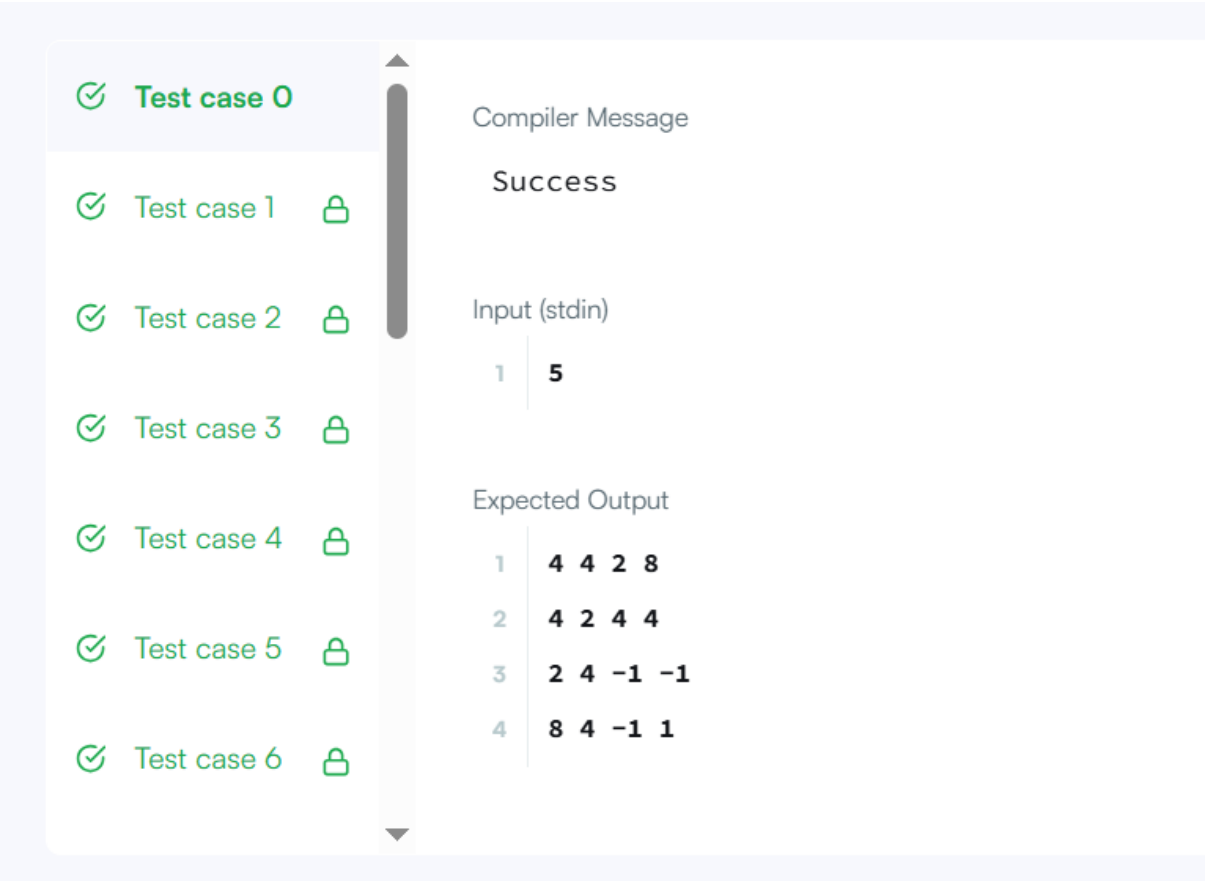
For example:

- **KnightL(3,3)** can only move back and forth between **(0,0)** and **(3,3)**
- Therefore it will **never reach (4,4)**.

Complete JAVA Code:

```
import java.util.*;

public class Solution {

    static int bfs(int n, int a, int b) {

        int[][] moves = {

            {a,b},{a,-b},{-a,b},{-a,-b},

            {b,a},{b,-a},{-b,a},{-b,-a}

        };

        boolean[][] visited = new boolean[n][n];

        Queue<int[]> queue = new LinkedList<>();
```

```
    queue.add(new int[]{0,0,0});
    visited[0][0] = true;
    while(!queue.isEmpty()) {
        int[] cur = queue.poll();
        int x = cur[0];
        int y = cur[1];
        int dist = cur[2];
        if(x == n-1 && y == n-1)
            return dist;
        for(int[] m : moves) {
            int nx = x + m[0];
            int ny = y + m[1];
            if(nx >= 0 && ny >= 0 && nx < n && ny < n && !visited[nx][ny]) {
                visited[nx][ny] = true;
                queue.add(new int[]{nx, ny, dist + 1});
            }
        }
    }
    return -1;
}
public static void knightlOnAChessboard(int n) {
    int[][] result = new int[n-1][n-1];
    for(int a = 1; a < n; a++) {
        for(int b = 1; b < n; b++) {
            result[a-1][b-1] = bfs(n, a, b);
        }
    }
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-1; j++) {
```

```
            System.out.print(result[i][j] + " ");
        }
        System.out.println();
    }
  }
 public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    knightlOnAChessboard(n);
  }
}
```

Observation/Solution: